SIMULADOR DE MEMÓRIA VIRTUAL

1. Conceitos básicos

O simulador é uma página web que mostra, passo a passo, como o sistema operacional gerencia a memória virtual: como um endereço virtual é traduzido, quando ocorre um page fault, qual página é removida da memória quando ela enche e para onde ela vai. A página é um arquivo único. Não precisa de instalação, de internet nem de servidor.

2. Atividade prática

2.1 Ferramentas necessárias

- Git: https://git-scm.com/install/
- Após a instalação, certifique-se que o git está instalado usando o comando `git --version`

2.2 Execução do simulador

- No terminal, navegue até um diretório de sua preferência e clone o repositório:
  `git clone https://github.com/mmaria-alves/simulador-memoria-virtual.git`
- Acesse a pasta do simulador:
  `cd simulador-memoria-virtual/simulador`
- Pelo terminal, execute o arquivo:
  `start vm_visual.html`
  Obs.: o arquivo também pode ser executado dando dois cliques nele no Explorador de Arquivos.

2.3 Conhecendo a barra de controles

- Input: escolhe qual conjunto de instruções será simulado (`in1`, `in3`, `in4`, `in7` ou `in8`).
- Frames: escolhe quantos quadros de memória física existem (8, 16 ou 32).
- Algorithm: escolhe o algoritmo de substituição de páginas (FIFO, Random, Clock, ESC/NRU, Aging ou Working Set).
- Load: aplica as três escolhas acima e reinicia a simulação.
  - Atenção: trocar uma opção não muda nada até você clicar em Load.
- Botões de navegação: ⏮ vai ao primeiro passo, ◂ volta, ▸ avança, ⏭ vai direto ao último passo. O botão Play roda tudo automaticamente.
- Contador (ex.: 3 / 31): passo atual e total de passos da simulação.
- Speed: controla a velocidade do Play (1 = lento, 10 = rápido).

2.4 Escolhendo e carregando uma simulação

- No topo da página, escolha:
    - Input: in1 (1 processo, 30 instruções)
    - Frames: 8
    - Algorithm: FIFO
- Clique em Load.
    - Resultado esperado: o contador do topo mostra 1 / 31, o painel CPU & MMU mostra `ctx-sw to P0` (o sistema passou a executar o processo P0) e a memória física mostra 0 / 8 frames used.
- Neste exemplo, o processo P0 vai acessar páginas de memória uma por uma, e você vai acompanhar o que o sistema operacional faz a cada acesso.
- Clique no botão de próximo cinco vezes, lendo cada frase do rodapé. O primeiro acesso é a leitura da página 14 (`read VP14`).
    - “Read: P0 accesses VP14 (0x0E000)”: o processo pede o endereço virtual 0x0E000.
    - “Page fault: VP14 is not in RAM”: a página não está na memória, acontece um page fault.
    - “Free frame: Frame 0 is available. No eviction needed”: há um quadro livre, então ninguém precisa sair.
    - “Zero: Frame 0 cleared”: o quadro é preenchido com zeros, porque é uma página nova.
    - “Mapped: P0:VP14 is now in RAM at Frame 0”: a página foi carregada e registrada na tabela de páginas.
- Observe o que mudou na tela:
    - CPU & MMU: o endereço virtual foi dividido em número da página (VPN) e deslocamento (offset), e o selo “Page fault (loaded)” apareceu.
    - Page Table: a linha da página 14 agora tem Status In RAM e Frame F00.
    - Physical Memory: o quadro F00 mostra P0 : VP14, e o contador passou para 1 / 8 frames used.
    - Resultado esperado no rodapé: Maps = 1, Zeros = 1, Page faults = 1.

2.5 Modificando a página

- Avance para o passo 3 (`write VP51`).
    - O processo agora escreve na página 51. Ela também sofre page fault e é carregada no quadro 1;
    - A frase “Write completes. Page marked modified (dirty)” deve aparecer na tela;
- Observe o que mudou na tela:
    - Na tabela de páginas, a coluna M da página 51 acende.

    - Na memória física, o quadro `F01` ganha uma bolinha amarela que indica página alterada.

2.6 Preenchendo a memória

- Continue avançando até o passo 9.
- A cada passo, uma página nova é carregada no próximo quadro livre: páginas 15, 55, 43, 10, 11 e 37 ocupam os quadros de 2 a 7
- No passo 9, o contador chega a **8 / 8 frames used**, ou seja, a memória está cheia.
    - Resultado esperado no rodapé: Maps = 8, Page faults = 8, Evicts = 0.

2.7 Memória cheia e substituição de páginas

- Avance para o passo 10 (`write VP20`). Não há mais quadros livres.
    - “Evict: No free frames. FIFO (oldest page) selects Frame 0 (holds P0:VP14)”: o FIFO remove a página que entrou primeiro, a 14.
    - “Clean: P0:VP14 was unmodified. Frame discarded. Disk copy is still valid”: como ela não foi modificada, o quadro é simplesmente descartado, sem gravar nada no disco.
    - Depois a página 20 deve ser carregada no quadro 0.
- Avance para o passo 11 (`read VP41`). O FIFO deve remover a página 51, que está no quadro 1.
    - “Out: P0:VP51 was modified. Written to swap to preserve changes”: como a página 51 foi modificada no passo 3, ela precisa ser gravada no disco (swap) antes de sair.
- Observe as mudanças na tela:
    - Na área Disk Storage, o disco gira, o braço se move e o Swap Space passa a mostrar 1 page, com um bloco `P0·VP51`.
    - No rodapé, o resultado deve ser Evicts = 2 e Outs = 1.
- Em resumo, no algoritmo FIFO, quando a memória enche, o sistema remove a página mais antiga. Se ela foi modificada, vai para o disco. Se não foi, é só descartada.

2.8 E se a página removida ainda for necessária?

- Avance até o passo 15 (`read VP15`).
    - A página 15 foi carregada até o passo 4, removida no passo 12. e agora é pedida de novo. Resultado esperado: novo page fault e mais uma remoção.
- O mesmo acontece no passo 19 com a página 37: ela foi carregada no passo 9, removida no 17 e pedida de novo no passo 19.

2.9 Uma página volta do disco e o único acerto

- Avance até o passo 24 (`write VP38`).
    - A página 38 foi modificada no passo 12 e enviada ao swap no passo 20. Neste passo, ela é pedida de novo.
    - “In: P0:VP38 restored from swap into Frame 6”: ela é lida do disco de volta para a memória. No rodapé, Ins passa a 1.
- Avance até o passo 28 (`write VP05`).
    - “Hit: VP5 is already in RAM at Frame 1. No disk access needed”: é o **primeiro acesso sem page fault** da simulação. A página 5 acabou de ser carregada no passo anterior e ainda está na memória.
    - O selo da TLB passa a mostrar **1/27 Hits**, e o rodapé não incrementa Page faults.

2.10 Resultados finais

- Avance até o último passo.
    - Resultado esperado no rodapé: Maps = 29, Evicts = 21, Ins = 1, Outs = 6, Zeros = 28, Segv = 0, Context switch = 1, Page faults = 29, Total cost = 42,820.
- Dos 30 acessos à memória, 29 geraram page fault. A memória de 8 quadros é pequena para as páginas que esse processo usa.